

Εργαστήριο 5 - Πολυμορφισμός (Polymorphism) & Virtual Methods

Στο πέμπτο και τελευταίο εργαστήριο θα επεκτείνουμε το σύστημα κλάσεων που αναπτύξατε στα προηγούμενα εργαστήρια εισάγοντας την έννοια του πολυμορφισμού (polymorphism). Στη C++ ο πολυμορφισμός επιτρέπει σε ένα pointer ή reference προς μια βασική κλάση να μπορεί να «δείχνει» σε αντικείμενα παράγωγων κλάσεων και να καλεί τη σωστή έκδοση των συναρτήσεων που έχουν γίνει override. Με αυτόν τον τρόπο μπορούμε να χειριστούμε διαφορετικούς τύπους βιολογικών ακολουθιών (DNA, RNA, Protein) με έναν ενιαίο τρόπο, χρησιμοποιώντας κοινές διεπαφές.

Ξεκινώντας από τις κλάσεις του Εργαστηρίου 4, θα μετατρέψετε τη βασική κλάση Sequence σε abstract class προσθέτοντας τουλάχιστον δύο pure virtual μεθόδους. Οι μέθοδοι describe() και isValid() θα γίνουν pure virtual. Η κλάση Sequence ως abstract δεν θα μπορεί πλέον να δημιουργήσει αντικείμενα. Θα δημιουργούνται μόνο αντικείμενα των παράγωγων κλάσεων DNASquence, RNASquence και ProteinSequence. Ωστόσο, θα μπορείτε να χρησιμοποιείτε pointers (Sequence*) και references (Sequence&) προς Sequence, οι οποίοι θα δείχνουν σε αντικείμενα των παράγωγων κλάσεων και θα ενεργοποιούν τη σωστή εκδοχή των overridden μεθόδων.

Για παράδειγμα, ο κώδικας:

```
Sequence* s1 = new DNASquence("ACGTACGT");  
Sequence* s2 = new RNASquence("AUGGCUAG");  
Sequence* s3 = new ProteinSequence("MTAPQL");  
  
s1->describe(); // καλείται DNASquence::describe  
s2->describe(); // καλείται RNASquence::describe  
s3->describe(); // καλείται ProteinSequence::describe
```

πρέπει να καλεί αντίστοιχα τις describe() των DNASquence, RNASquence και ProteinSequence. Για να συμβεί αυτό, θα κάνετε τις μεθόδους describe() των

παράγωγων κλάσεων virtual και θα υλοποιήσετε overriding, ώστε κάθε κλάση να εμφανίζει τη δική της εξειδικευμένη μορφή περιγραφής. Θα προσθέσετε επίσης virtual destructor στη βασική κλάση, ώστε η καταστροφή αντικειμένων μέσω base pointer να γίνεται σωστά, αποτρέποντας memory leaks και εξασφαλίζοντας την ορθή ακολουθία destructors.

Θα προσθέσετε τη virtual μέθοδο isValid() στη βασική κλάση Sequence και θα την υλοποιήσετε διαφορετικά σε κάθε παράγωγη κλάση. Για μια DNASequence η isValid() πρέπει να ελέγχει αν η ακολουθία περιέχει μόνο A, C, G και T· η RNASequence πρέπει να δέχεται A, C, G και U· ενώ η ProteinSequence πρέπει να ελέγχει αν οι χαρακτήρες ανήκουν στο σύνολο των επιτρεπτών αμινοξέων. Καλώντας την isValid() μέσω base pointer θα πρέπει να ενεργοποιείται αυτόματα η υλοποίηση της αντίστοιχης παράγωγης κλάσης. Για παράδειγμα:

```
cout << "Valid? " << s1->isValid() << endl;  
cout << "Valid? " << s2->isValid() << endl;  
cout << "Valid? " << s3->isValid() << endl;
```

πρέπει να εμφανίζει διαφορετικά αποτελέσματα ανάλογα με το περιεχόμενο και τον τύπο κάθε ακολουθίας.

Στη συνέχεια θα υλοποιήσετε ένα vector<Sequence*> μέσα στο οποίο θα αποθηκεύετε αντικείμενα διαφορετικών τύπων (DNASequence, RNASequence, ProteinSequence). Καλώντας describe() σε κάθε στοιχείο του vector, η C++ θα καλέσει αυτόματα την κατάλληλη έκδοση describe(), ανάλογα με τον πραγματικό τύπο του αντικειμένου στο οποίο δείχνει το pointer.

Στη συνάρτηση main() θα πρέπει να δημιουργήσετε αντικείμενα DNASequence, RNASequence και ProteinSequence, αλλά να τα χειριστείτε όλα μέσω pointers προς Sequence. Θα αποθηκεύσετε αυτά τα αντικείμενα σε ένα vector<Sequence*> και θα καλέσετε για καθένα τη describe(), την length() και την isValid(). Στη συνέχεια θα διαγράψετε όλα τα αντικείμενα και θα παρατηρήσετε την ορθή αλληλουχία καταστροφής χάρη στη χρήση του virtual destructor στη βάση. Τέλος, θα δημιουργήσετε ένα Gene όπως στο προηγούμενο εργαστήριο και θα προσθέσετε Isoforms με RNASequences. Καλώντας τη describe() του Gene θα πρέπει να

εμφανίζονται οι RNASequence μέσω της πολυμορφικής describe(). Θα παρατηρήσετε όλα τα μηνύματα κατασκευής και καταστροφής, ώστε να κατανοήσετε πλήρως τη σειρά δημιουργίας και διαχείρισης αντικειμένων σε ένα πολυμορφικό περιβάλλον.

Με αυτό το εργαστήριο θα αποκτήσετε ολοκληρωμένη κατανόηση της λειτουργίας του πολυμορφισμού, των virtual και override μεθόδων, της δυναμικής δέσμευσης (dynamic dispatch), της χρήσης abstract base classes και της σημασίας των virtual destructors στη C++. Θα δείτε πώς η C++ επιτρέπει την ενιαία και ευέλικτη αναπαράσταση διαφορετικών βιολογικών οντοτήτων μέσω κοινής ιεραρχίας κλάσεων, γεγονός που καθιστά τον κώδικα επεκτάσιμο, οργανωμένο και πιο κοντά σε πραγματικές βιοπληροφορικές εφαρμογές.

Στη συνάρτηση main() θα πρέπει να:

- δημιουργήσετε αντικείμενα DNASequence, RNASequence και ProteinSequence, αλλά να τα χειριστείτε μέσω Sequence*,
- αποθηκεύσετε όλα τα αντικείμενα σε ένα vector<Sequence*>,
- καλέσετε describe(), length() και isValid() για κάθε Sequence* μέσα στο vector,
- διαγράψετε τα αντικείμενα και να παρατηρήσετε τη σωστή αλληλουχία καταστροφής μέσω virtual destructor,
- δημιουργήσετε ένα Gene από το προηγούμενο εργαστήριο και να προσθέσετε Isoforms που περιλαμβάνουν RNASequence,
- καλέσετε τη describe() του Gene και να επιβεβαιώσετε ότι οι RNASequence εμφανίζονται μέσω της πολυμορφικής describe(),
- παρακολουθήσετε τα μηνύματα κατασκευής και καταστροφής ώστε να κατανοήσετε τη σειρά δημιουργίας και διαχείρισης αντικειμένων σε ένα

πολυμορφικό περιβάλλον.

Με αυτό το εργαστήριο θα κατανοήσετε σε βάθος τη λειτουργία του πολυμορφισμού, την έννοια των `virtual` και `override` μεθόδων, τη δυναμική δέσμευση (`dynamic dispatch`), τη χρήση `abstract base classes`, τη σημασία των `virtual destructors` και τον τρόπο με τον οποίο η C++ επιτρέπει την ενιαία αναπαράσταση διαφορετικών τύπων βιολογικών οντοτήτων μέσω μιας κοινής ιεραρχίας κλάσεων.